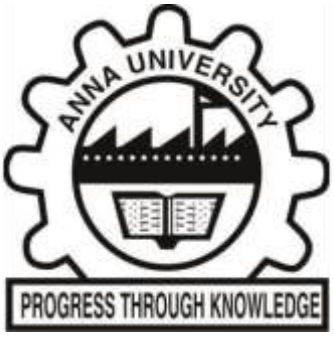


COLLEGE OF ENGINEERING GUINDY,
ANNA UNIVERSITY,

CHENNAI-600025.

ELECTRONICS AND COMMUNICATION ENGINEERING

EC23C14 DIGITAL VLSI DESIGN

TASK REPORT

TOPIC: PERFORM ASIC DESIGN FLOW AND PERFORM TIMING ANALYSIS FOR CARRY LOOK AHEAD ADDER AND BOOTH MULTILPLIER

Submitted by

SAABIQ U A – 2023105514

KISHORE R – 2023105522

HARIHARAN S – 2023105528

ABSTRACT

This project presents the design and implementation of a high-speed multiplier using the Radix-4 Modified Booth algorithm combined with a Carry Lookahead Adder (CLA). Multipliers are fundamental components in digital signal processing and processor architectures, where performance is often limited by propagation delay. To address this, the Modified Booth algorithm is employed to reduce the number of partial products, thereby improving computational efficiency.

The generated partial products are compressed using Carry Save Adders (CSA), and the final addition is performed using a Carry Lookahead Adder to achieve faster carry propagation. The design is implemented using Verilog HDL and functionally verified through simulation in Vivado. Performance metrics such as area, delay, and power are analyzed.

Further, the design is taken through a complete ASIC design flow using both open-source tools with the Sky130 nm PDK and industry-standard Cadence tools (Genus and Innovus) with a 45 nm library. The results demonstrate the effectiveness of the proposed architecture in achieving improved speed while maintaining reasonable area and power characteristics.

INTRODUCTION

Multiplication is one of the most critical arithmetic operations in digital systems, widely used in applications such as digital signal processing (DSP), image processing, and microprocessor design. The performance of these systems is often constrained by the speed and efficiency of the multiplier unit. Conventional multiplication techniques suffer from increased delay due to the generation and accumulation of a large number of partial products.

To overcome these limitations, advanced multiplication techniques such as the Modified Booth algorithm are used. The Radix-4 Modified Booth encoding reduces the number of partial products by encoding the multiplier bits in groups, thereby decreasing the overall computation time. However, the efficiency of the multiplier also depends on how these partial products are added.

In this work, Carry Save Adders (CSA) are used to efficiently compress multiple partial products without propagating carries at each stage. The final summation is performed using a Carry Lookahead Adder (CLA), which significantly reduces carry propagation delay compared to conventional ripple carry adders.

The complete design is described using Verilog HDL and verified through simulation. To evaluate real-world feasibility, the design is implemented through a full RTL to GDSII flow using both open-source and commercial Electronic Design Automation (EDA) tools. This provides insights into the trade-offs between delay, area, and power across different design environments and technologies.

PROPOSED DESIGN

The proposed design implements a high-speed signed multiplier using the Radix-4 Modified Booth algorithm combined with a Carry Lookahead Adder (CLA). The architecture is optimized to reduce the number of partial products and improve overall computational speed. The multiplier takes two signed 8-bit inputs and produces a 16-bit product along with an overflow indication. The overall architecture consists of four main stages: Booth encoding, partial product generation, carry save addition, and final carry lookahead addition.

2.1 Architecture Overview

The input operands are first processed using a Radix-4 Booth encoder, which reduces the number of partial products by encoding the multiplier bits in groups of three. Based on this

encoding, the Partial Product Generator (PPG) produces appropriately shifted and signed partial products.

These partial products are then added using Carry Save Adders (CSA) in two stages. The CSA structure allows multiple operands to be added simultaneously without propagating carries, thereby reducing intermediate delay. Finally, a 16-bit Carry Lookahead Adder (CLA) is used to combine the final sum and carry outputs from the CSA stages, producing the final product and overflow signal.

2.2 Booth Encoder

The Booth encoder examines the multiplier bits in overlapping groups of three and generates control signals that determine whether the multiplicand should be added, subtracted, or ignored. This encoding reduces the number of partial products by approximately half compared to conventional multiplication methods, thereby improving efficiency.

2.3 Partial Product Generator

The Partial Product Generator (PPG) generates signed and shifted versions of the multiplicand based on the Booth encoding outputs. It produces four partial products corresponding to the Radix-4 encoding scheme. Proper sign extension is handled to ensure correct signed multiplication.

2.4 Carry Save Adder (CSA)

Two stages of Carry Save Adders are used to compress the partial products. In the first stage, three partial products are added to produce intermediate sum and carry outputs. In the second stage, these outputs are combined with the remaining partial product. The CSA structure avoids carry propagation at each stage, significantly reducing delay.

2.5 Carry Lookahead Adder (CLA)

The final addition is performed using a 16-bit Carry Lookahead Adder. The CLA improves performance by computing carry signals in parallel using generate and propagate logic, unlike ripple carry adders which propagate carry sequentially. This results in faster computation of the final product. Overall, the proposed architecture effectively balances speed and hardware complexity by combining efficient partial product reduction with fast addition techniques.

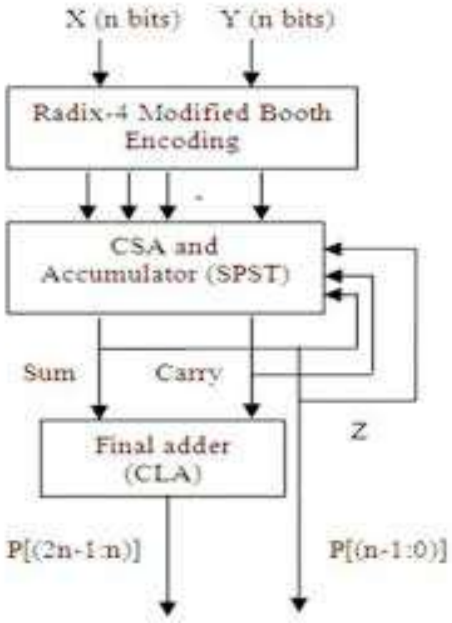


Figure 2.1: Architecture of Radix-4 Modified Booth Multiplier with CLA

3.RTL DESIGN AND SIMULATION

3.1 RTL Design Description

The proposed Radix-4 Modified Booth multiplier with Carry Lookahead Adder (CLA) is implemented using Verilog HDL in a modular and hierarchical manner. The design processes two signed 8-bit inputs and produces a 16-bit signed product along with an overflow signal.

The overall architecture consists of four major functional blocks: Booth encoder, Partial Product Generator (PPG), Carry Save Adder (CSA), and Carry Lookahead Adder (CLA). The Booth encoder generates control signals based on the multiplier input, which are then used by the PPG to generate four partial products. These partial products are reduced using two stages of CSA to produce intermediate sum and carry outputs. Finally, a 16-bit CLA is used to compute the final product efficiently by reducing carry propagation delay.

3.2 Top Module Implementation

The top module integrates all submodules and defines the complete data path of the multiplier. It connects the Booth encoder, partial product generator, CSA stages, and CLA to form a complete multiplication unit.

The inputs X and Y are 8-bit signed operands. The Booth encoder processes input X to generate encoding signals, which control the generation of partial products from input Y in the PPG block. The generated partial products (PP0, PP1, PP2, PP3) are then passed through two stages of Carry Save Adders. The first CSA stage reduces three partial products into intermediate sum and carry signals, while the second stage combines these with the remaining

partial product. The outputs of the CSA stage are shifted appropriately and provided as inputs to the 16-bit Carry Lookahead Adder. The CLA performs the final addition and produces the 16-bit product along with a carry-out signal, which is used as the overflow indicator.

The Verilog implementation of the top module is shown below.

```
module Booth_CLA_Top (
    input  signed [7:0] X,
    input  signed [7:0] Y,
    output signed [15:0] Product,
    output  Overflow
);
    wire [11:0] op;
    wire signed [15:0] PP0, PP1, PP2, PP3;
    wire [15:0] S1, C1;
    wire [15:0] S2, C2;
    // Booth Encoder
    booth_4 BE (.X(X), .op(op));

    // Partial Products
    ppg PPG ( .Y(Y), .op(op), .PP0(PP0), .PP1(PP1), .PP2(PP2),  .PP3(PP3) );
    // CSA Stage 1
    csa CSA1 (.A(PP0), .B(PP1), .C(PP2), .Sum(S1), .Carry(C1));
    // CSA Stage 2
    csa CSA2 (.A(S1), .B(C1 << 1), .C(PP3), .Sum(S2), .Carry(C2));
    // Final Adder
    CLA_16 CLA (.A(S2), .B(C2 << 1), .Sum(Product), .Cout(Overflow));
endmodule
```

3.3 Simulation Results

The functional correctness of the proposed Radix-4 Modified Booth multiplier with CLA is verified using a testbench in Vivado. Various combinations of signed input values are applied to validate the multiplier operation for both positive and negative numbers.

From the simulation waveform, it can be observed that the design correctly computes the product for different input conditions. A few sample test cases are listed below:

- For X = 5 and Y = 3, the output Product = 15, which is correct.
- For X = -5 and Y = 3, the output Product = -15, validating signed multiplication.
- For X = 127 and Y = 127, the output Product = 16129, which matches the expected result.
- For X = -128 and Y = -128, the output Product = 16384, confirming correct handling of negative inputs.
- For X = -128 and Y = 127, the output Product = -16256, which is also correct.

The waveform demonstrates proper operation of all internal stages, including Booth encoding, partial product generation, and carry save addition. The final output obtained through the Carry Lookahead Adder matches the expected mathematical results. The overflow signal remains low for the tested cases, indicating that the output values are within the representable range of the design. The simulation results confirm that the proposed design performs accurate signed multiplication across a wide range of input values.



Figure 3.1: Simulation waveform showing correct operation of Radix-4 Booth Multiplier with CLA

4.SYNTHESIS RESULTS (VIVADO)

The proposed Radix-4 Modified Booth multiplier with Carry Lookahead Adder (CLA) is synthesized using Xilinx Vivado. The synthesis results provide insights into resource utilization, timing performance, and power consumption of the design.

4.1 Resource Utilization

The synthesized design utilizes FPGA resources efficiently. The utilization details are summarized below:

- Look-Up Tables (LUTs): 93 out of 20800 (0.45%)
- I/O Ports: 33 out of 106 (31.13%)

The results indicate that the design occupies a very small portion of the available logic resources, demonstrating an area-efficient implementation.

Resource	Utilization	Available	Utilization %
LUT	93	20800	0.45
IO	33	106	31.13

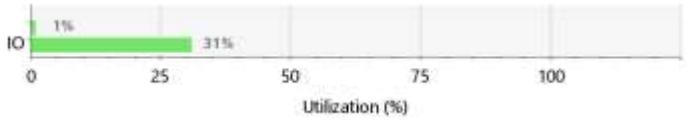


Figure 4.1: FPGA Resource Utilization of Booth CLA Multiplier in Vivado

4.2 Timing Analysis

Timing analysis is performed to evaluate the speed and reliability of the design. The key timing parameters obtained from the report are:

- Worst Negative Slack (WNS): 6.075 ns
- Total Negative Slack (TNS): 0.000 ns
- Worst Hold Slack (WHS): 1.375 ns

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	6.075 ns	Worst Hold Slack (WHS):	1.375 ns	Worst Pulse Width Slack (WPWS):	NA
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	NA
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	NA
Total Number of Endpoints:	17	Total Number of Endpoints:	17	Total Number of Endpoints:	NA

All user specified timing constraints are met.

Figure 4.2: Timing Summary Report Showing Setup and Hold Slack

The critical path delay is observed to be approximately **13.925 ns**, corresponding to the path from input Y[4] to output Product[13].

Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock	Exception
Path 11	1.375	3	2	28	X[1]	Product[2]	2.375	1.327	1.047	1.0	input port clock		MinDelay Path 1.000ns
Path 12	1.424	3	2	14	X[0]	Product[0]	2.424	1.327	1.097	1.0	input port clock		MinDelay Path 1.000ns
Path 13	1.454	3	2	28	X[1]	Product[3]	2.454	1.399	1.055	1.0	input port clock		MinDelay Path 1.000ns
Path 14	1.473	3	2	28	X[1]	Product[1]	2.473	1.383	1.090	1.0	input port clock		MinDelay Path 1.000ns
Path 15	1.593	4	3	28	X[1]	Product[5]	2.593	1.373	1.220	1.0	input port clock		MinDelay Path 1.000ns
Path 16	1.628	4	3	12	X[7]	Product[14]	2.628	1.359	1.268	1.0	input port clock		MinDelay Path 1.000ns
Path 17	1.638	5	4	12	X[7]	Overflow	2.638	1.474	1.165	1.0	input port clock		MinDelay Path 1.000ns
Path 18	1.660	5	4	12	X[7]	Product[15]	2.660	1.477	1.184	1.0	input port clock		MinDelay Path 1.000ns
Path 19	1.689	5	4	12	X[4]	Product[12]	2.689	1.421	1.268	1.0	input port clock		MinDelay Path 1.000ns
Path 20	1.711	4	3	14	X[6]	Product[8]	2.711	1.441	1.270	1.0	input port clock		MinDelay Path 1.000ns

Figure 4.3: Setup Timing Analysis Showing Critical Path Delay

Name	Slack ^1	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock	Exception
Path 11	1.375	3	2	28	X[1]	Product[2]	2.375	1.327	1.047	1.0	input port clock		MinDelay Path 1.000ns
Path 12	1.424	3	2	14	X[0]	Product[0]	2.424	1.327	1.097	1.0	input port clock		MinDelay Path 1.000ns
Path 13	1.454	3	2	28	X[1]	Product[3]	2.454	1.399	1.055	1.0	input port clock		MinDelay Path 1.000ns
Path 14	1.473	3	2	28	X[1]	Product[1]	2.473	1.383	1.090	1.0	input port clock		MinDelay Path 1.000ns
Path 15	1.593	4	3	28	X[1]	Product[5]	2.593	1.373	1.220	1.0	input port clock		MinDelay Path 1.000ns
Path 16	1.628	4	3	12	X[7]	Product[14]	2.628	1.359	1.268	1.0	input port clock		MinDelay Path 1.000ns
Path 17	1.638	5	4	12	X[7]	Overflow	2.638	1.474	1.165	1.0	input port clock		MinDelay Path 1.000ns
Path 18	1.660	5	4	12	X[7]	Product[15]	2.660	1.477	1.184	1.0	input port clock		MinDelay Path 1.000ns
Path 19	1.689	5	4	12	X[4]	Product[12]	2.689	1.421	1.268	1.0	input port clock		MinDelay Path 1.000ns
Path 20	1.711	4	3	14	X[6]	Product[8]	2.711	1.441	1.270	1.0	input port clock		MinDelay Path 1.000ns

Figure 4.4: Hold Timing Analysis Showing Minimum Path Delay

Further analysis shows:

- Logic Delay: 5.305 ns
- Routing (Net) Delay: 8.620 ns

It is evident that routing delay contributes more significantly than logic delay to the total path delay. All timing constraints are satisfied, indicating that the design operates correctly within the specified clock period.

4.3 Power Analysis

Power analysis is carried out to estimate the total on-chip power consumption of the design. The results are as follows:

- Total On-Chip Power: 0.078 W
- Dynamic Power: 0.008 W
- Static Power: 0.070 W

Among the dynamic components, I/O power contributes the highest portion. The majority of total power consumption is due to static power, while dynamic power remains relatively low.

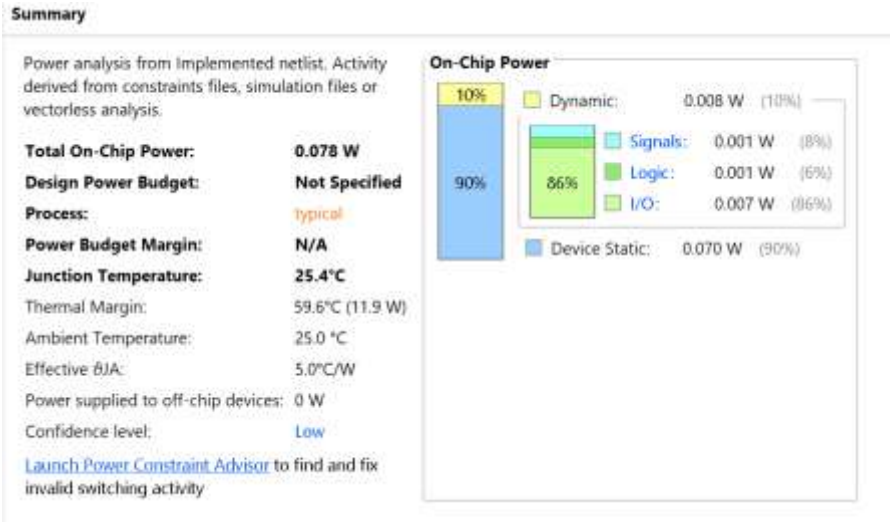


Figure 4.5: Power Consumption Analysis of the Design in Vivado

5. ASIC IMPLEMENTATION USING SKY130 (OPEN-SOURCE FLOW)

The proposed Radix-4 Modified Booth multiplier with Carry Lookahead Adder (CLA) is implemented using an open-source ASIC design flow with the Sky130 nm Process Design Kit (PDK). This flow converts the RTL design into a manufacturable layout through multiple stages including synthesis, floorplanning, placement, routing, and layout verification.

5.1 Synthesis

The RTL design is synthesized using Yosys, generating a gate-level netlist mapped to Sky130 standard cells. The synthesized design consists of approximately **505 standard cells with a total cell area of 3968.81 µm².** The design predominantly utilizes NAND gates, multiplexers, and inverters, indicating efficient implementation of arithmetic operations.

5.2 Floorplanning

In the floorplanning stage, the physical dimensions of the chip are defined along with placement of I/O pins and power distribution.

- Die Dimensions: 126.06 µm × 136.78 µm
- Die Area: 17241.4 µm²

The floorplan provides the foundation for subsequent placement and routing stages.

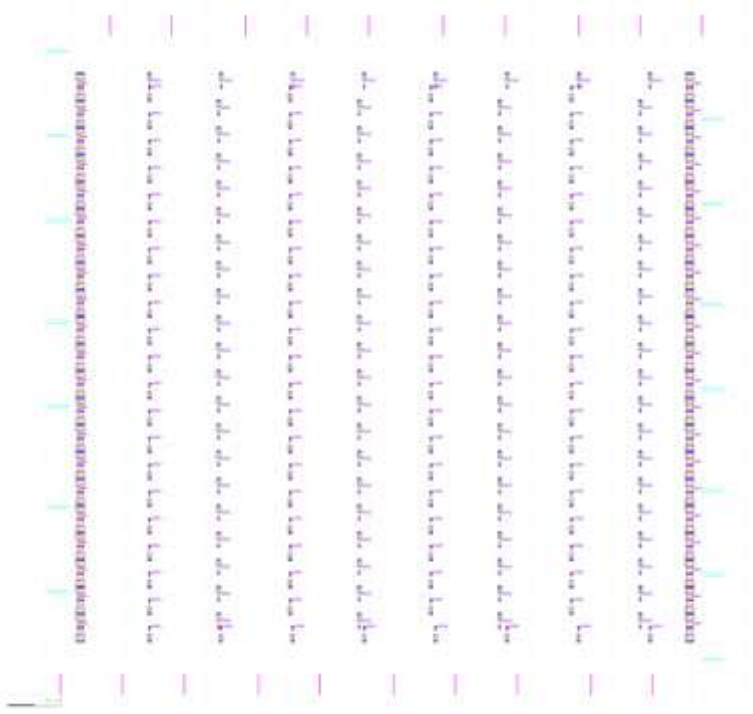


Figure 5.1: Floorplan of Booth CLA Multiplier using Sky130

5.3 Placement

During placement, standard cells are arranged within the defined core area to optimize wirelength and timing.

- Core Area: 13137.6 µm²
- Cell Area: 3968.81 µm²
- Utilization: 31.49%
- Total Instances: 765
- Standard Cells: 505

The moderate utilization ensures sufficient routing space and reduces congestion, improving overall design performance.

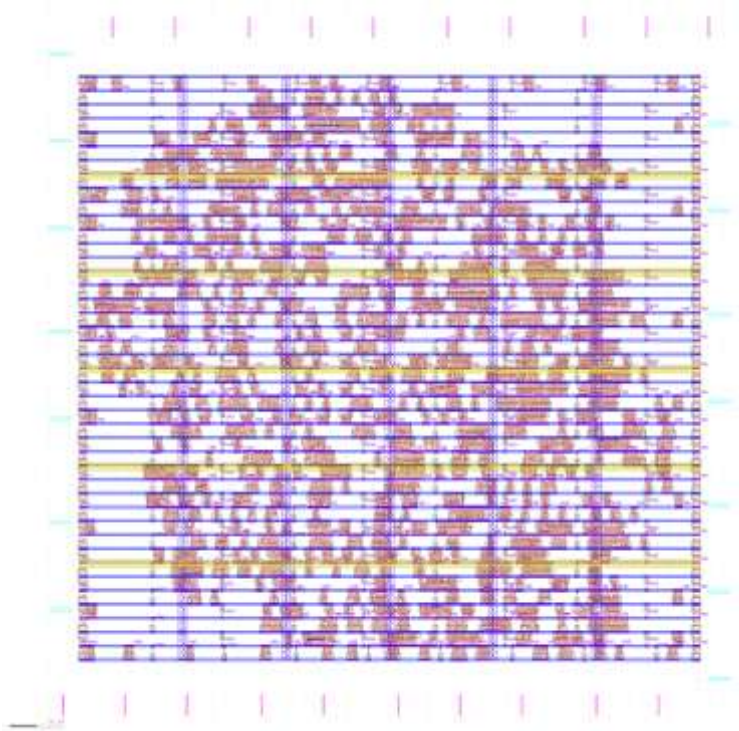


Figure 5.2: Placement of Standard Cells in the Design

5.4 Routing

Routing establishes connections between all placed cells using multiple metal layers.

- Routing Status: Successful
- DRC Violations: 0
- Total Wire Length: 12842 µm
- Total Vias: 3553
- Metal Layers Used: met1, met2, met3

The routing process was completed successfully with zero design rule violations, ensuring a physically valid design.

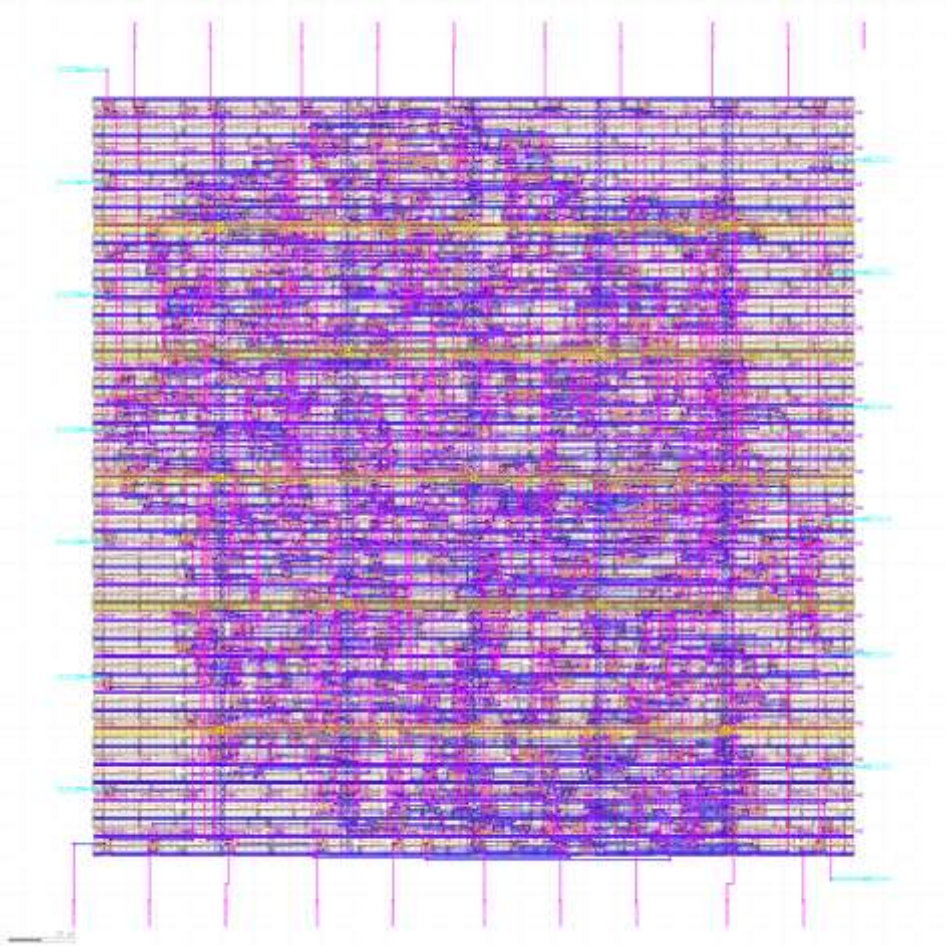


Figure 5.3: Routing of Booth CLA Multiplier

5.5 Timing Analysis Across Design Stages

The timing performance of the design is analyzed across different stages of the ASIC flow.

- Post-Synthesis Delay: 12.84 ns
- Post-Placement Delay: 11.68 ns
- Post-Routing Delay: 11.96 ns

The delay decreases after placement due to optimization techniques such as buffering and cell resizing. After routing, the delay slightly increases due to interconnect parasitics. However, the design maintains positive setup and hold slack at all stages, indicating successful timing closure.

This analysis highlights the impact of physical design on timing and demonstrates the effectiveness of the optimization techniques applied during the flow.

5.6 Final Layout (GDSII)

After routing, the final layout is generated and verified using Design Rule Check (DRC) and Layout Versus Schematic (LVS). The layout represents the physical implementation of the design, ready for fabrication.

The absence of DRC violations confirms that the layout adheres to manufacturing rules.

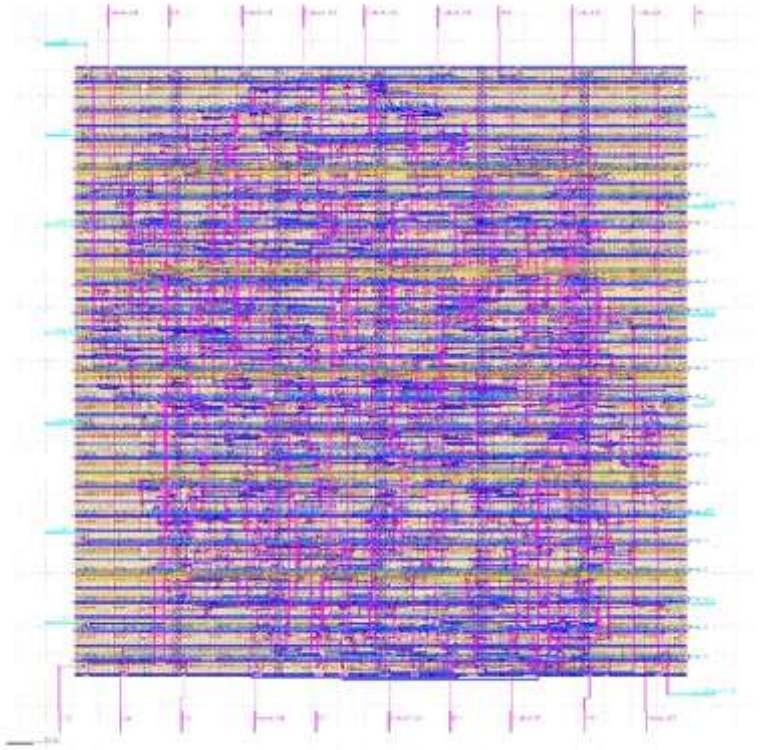


Figure 5.4: Final GDSII Layout of Booth CLA Multiplier

6. ASIC IMPLEMENTATION USING CADENCE TOOLS (GENUS & INNOVUS)

The proposed Radix-4 Modified Booth multiplier with Carry Lookahead Adder (CLA) is further implemented using Cadence Genus and Innovus tools to validate the design in an industry-standard ASIC design environment. A 45 nm generic standard cell library is used for this implementation.

6.1 RTL Synthesis using Genus

The RTL design is synthesized using Cadence Genus to generate an optimized gate-level netlist. The design is verified for correctness, and no critical issues are observed during the synthesis stage.

The synthesis results are summarized below:

- Total Cell Count: 195
- Total Area: 1073.284 µm²
- Critical Path Delay: 1.278 ns
- Slack: 8.722 ns

The positive slack indicates that the design meets timing requirements with a significant margin under ideal (pre-layout) conditions.

6.2 Floorplanning

The synthesized netlist is imported into Cadence Innovus, where floorplanning defines the physical boundaries of the design.

- Design Boundary: (0, 0) to (46.2 µm, 44.08 µm)

The floorplan establishes the layout area and prepares the design for placement.

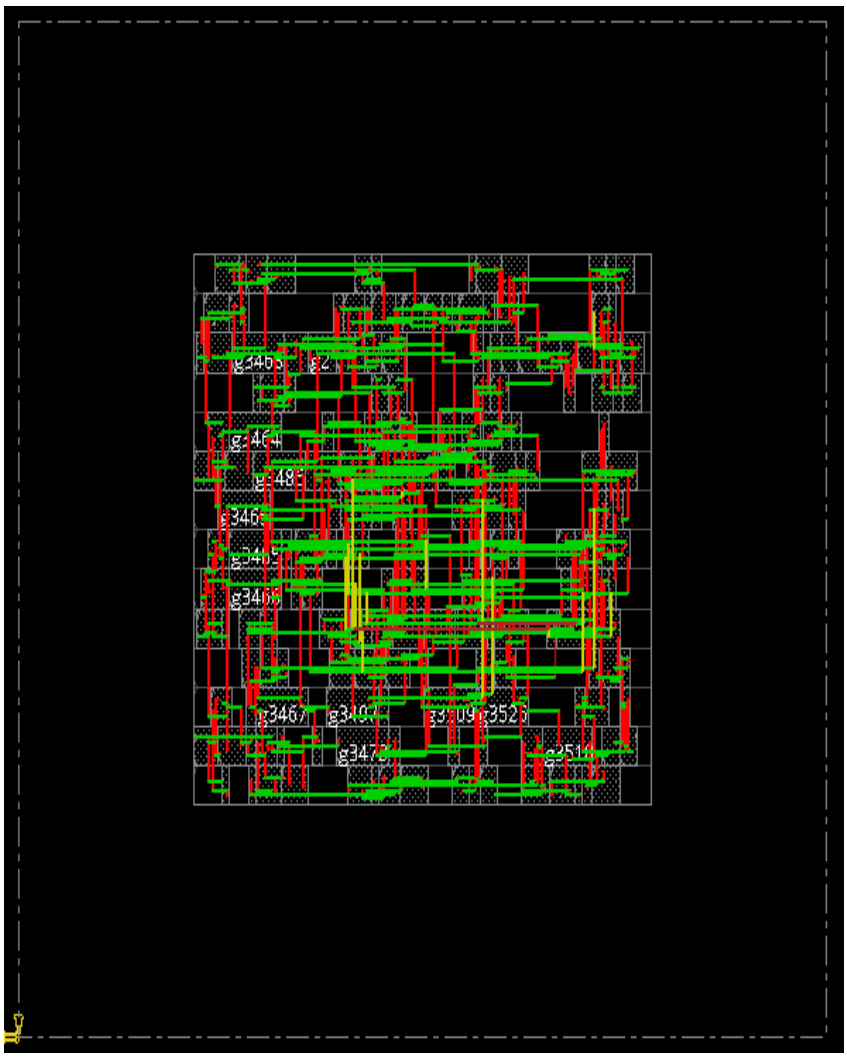


Figure 6.1: Floorplan of Booth CLA Multiplier in Cadence Innovus

6.3 Placement

During placement, standard cells are arranged within the core area to optimize wirelength and improve performance.

- Total Instances: 195
- Total Area after placement: 374.49 µm²

The placement ensures efficient utilization of area while maintaining sufficient routing space.

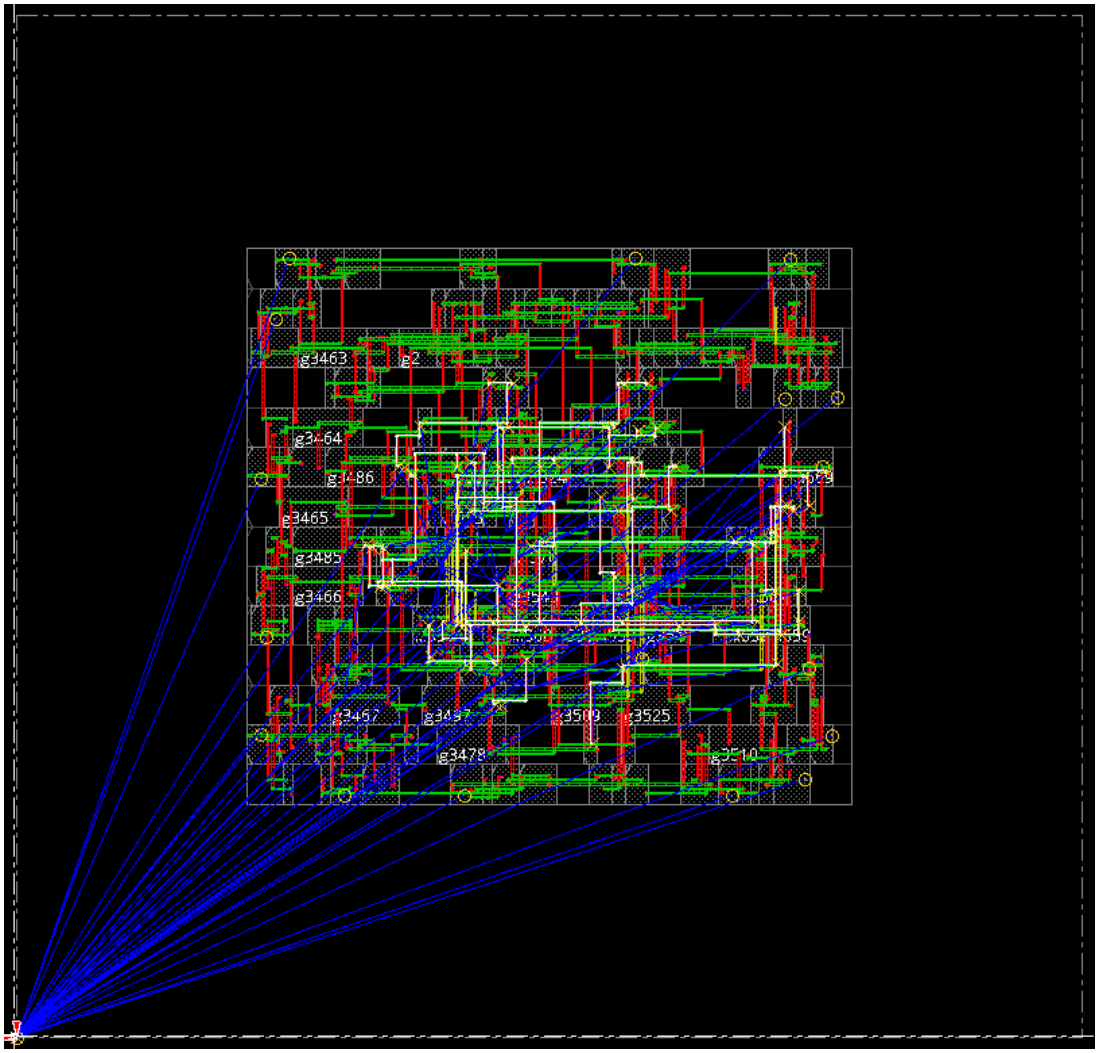


Figure 6.2: Placement of Standard Cells using Cadence Innovus

6.4 Routing

Routing connects all placed cells using available metal layers to ensure complete signal connectivity.

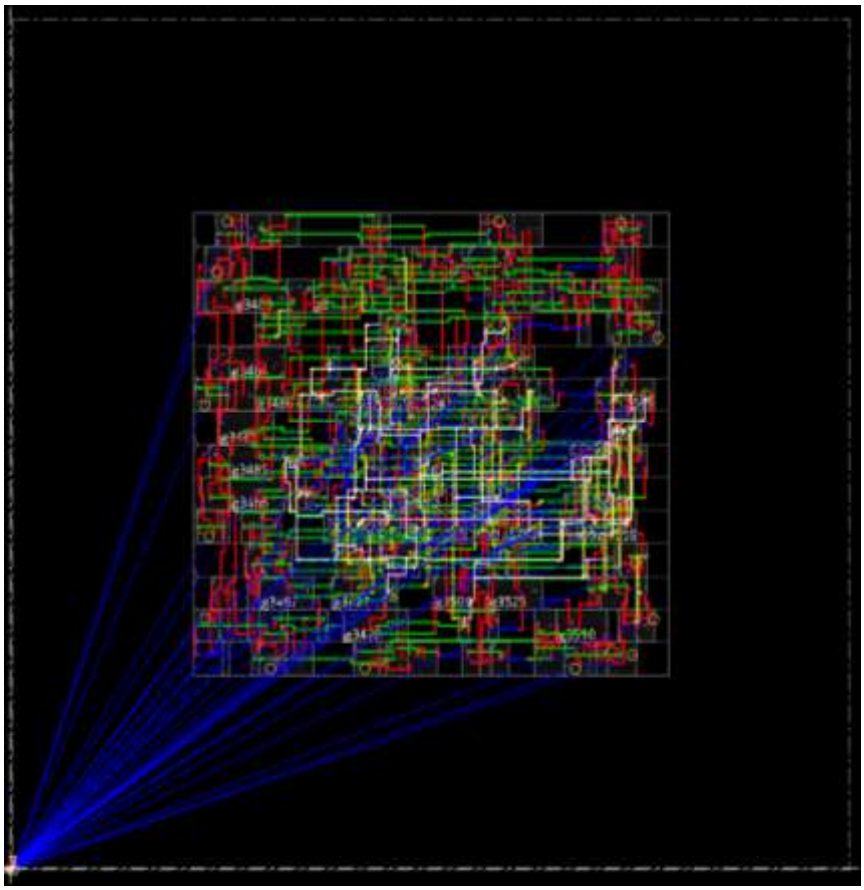


Figure 6.3: Routing of Booth CLA Multiplier in Cadence Innovus

6.5 Verification (DRC & Connectivity)

After routing, verification is performed to ensure physical correctness of the design.

- Connectivity Check: Passed
- DRC Violations: 0

The absence of DRC violations confirms that the design satisfies all design rules and is ready for fabrication.

6.6 Discussion

The Cadence implementation using 45 nm technology demonstrates significantly improved timing performance compared to FPGA implementation. The reduced delay is attributed to smaller feature size and optimized standard cell libraries However, the synthesis timing represents pre-layout conditions, and actual delays may increase after routing due to interconnect parasitics. Overall, the design achieves efficient area utilization and meets all timing and physical verification requirements.

7. RESULTS AND COMPARISON

The proposed Radix-4 Modified Booth multiplier with Carry Lookahead Adder (CLA) is implemented and analyzed across three different platforms: FPGA (Vivado), open-source ASIC flow using Sky130, and Cadence tools using a 45 nm technology library. The results obtained from each platform are compared in terms of area, timing, and power.

7.1 Comparison of Timing

The timing performance across different implementations is summarized below:

- Vivado (FPGA): 13.925 ns
- Sky130 (Post-Routing): 11.96 ns
- Cadence 45 nm (Post-Synthesis): 1.278 ns

The FPGA implementation exhibits higher delay due to programmable routing and configurable logic blocks. The Sky130 ASIC implementation shows improved timing performance due to dedicated standard cells and optimized routing. The Cadence implementation in 45 nm technology demonstrates the lowest delay, primarily due to technology scaling and reduced transistor dimensions.

7.2 Comparison of Area

The area comparison across different implementations is as follows:

- FPGA (Vivado): 93 LUTs (0.45% utilization)
- Sky130 ASIC: 3968.81 μm^2 (cell area)
- Cadence 45 nm: 374.49 μm^2

The FPGA implementation uses configurable logic resources, whereas ASIC implementations provide a more compact and optimized design. The Cadence implementation achieves the smallest area due to smaller technology node (45 nm) compared to Sky130 (130 nm).

7.3 Comparison of Power

The power consumption observed in FPGA implementation is:

- Total Power (Vivado): 0.078 W

The dynamic power is relatively low, while static power contributes a significant portion of the total power. ASIC implementations generally offer lower power consumption compared to FPGA; however, exact power values depend on switching activity and operating conditions.

7.4 Discussion

From the comparison, it is evident that ASIC implementations outperform FPGA implementations in terms of speed and area efficiency. The reduction in delay from FPGA to ASIC highlights the impact of dedicated hardware resources and optimized interconnects.

Furthermore, technology scaling from 130 nm (Sky130) to 45 nm (Cadence) significantly improves performance and reduces area. However, pre-layout timing results (Cadence synthesis) may differ from post-layout results due to interconnect parasitics. Overall, the proposed design demonstrates efficient performance across all platforms and validates its suitability for high-speed and area-constrained applications.

8. CONCLUSION

In this project, a Radix-4 Modified Booth multiplier integrated with a Carry Lookahead Adder (CLA) has been successfully designed, implemented, and analyzed through a complete RTL to GDSII flow. The design efficiently reduces the number of partial products using Booth encoding and improves addition speed through the use of a CLA, resulting in enhanced overall performance.

The functionality of the design is verified through simulation in Vivado, confirming correct operation for both positive and negative signed inputs. Synthesis results demonstrate efficient utilization of FPGA resources along with acceptable timing and power characteristics.

The design is further implemented using an open-source ASIC flow with the Sky130 nm PDK, where all stages including synthesis, floorplanning, placement, routing, and layout verification are successfully completed. The timing analysis across stages highlights the impact of physical design and interconnect parasitics on performance.

Additionally, the implementation using Cadence Genus and Innovus with a 45 nm technology library demonstrates significant improvements in delay and area due to technology scaling. The comparative analysis across FPGA and ASIC platforms validates the effectiveness of the proposed architecture in achieving high-speed and area-efficient multiplication.

Overall, the project provides a comprehensive understanding of digital design flow from RTL to physical layout and demonstrates the practical implementation of high-performance arithmetic circuits using both open-source and industry-standard tools.